ESPE
UNIVERSIDAD DE LAS FUERZAS ARMADAS
INNOVACIÓN PARA LA EXCELENCIA
1922 ECUADOR

DESARROLLO DE SOFTWARE SEGURO



Unidad 1 – Introducción y estudio de vulnerabilidades

Departamento de Ciencias de la Computación
Carrera de Software

Prof. Geovanny Cudco

TABLA DE CONTENIDOS — Unidad 1

Ruta de aprendizaje de la unidad

Temas que se abordarán desde la introducción hasta la seguridad en servidores web y bases de datos.



1 **1.1 INTRODUCCIÓN**
Visión general de la unidad, objetivos y alcance de los temas a tratar.

2 **1.1.1 QUÉ ES LA SEGURIDAD DEL SOFTWARE**
Concepto, principios y características fundamentales.

3 **1.1.2 LA IMPORTANCIA DE LA SEGURIDAD EN EL SOFTWARE**
Beneficios, impacto y razones para integrar la seguridad en el ciclo de vida del software.

4 **1.1.3 LAS VULNERABILIDADES Y SUS COSTES**
Tipos de vulnerabilidades, ejemplos y consecuencias económicas y operativas.

5 **1.1.4 GESTIÓN DE RIESGOS DE SEGURIDAD DEL SOFTWARE**
Identificación y análisis de riesgos, probabilidad e impacto, y toma de decisiones.

6 **1.1.5 PROPIEDADES DEL SOFTWARE SEGURO Y RESILIENTE**
Confidencialidad, integridad, disponibilidad, autenticación y no repudio.

7 **1.1.6 CONCEPTOS DE SEGURIDAD**
Activos, amenazas, vulnerabilidades, riesgos y controles en el software.

8 **1.1.7 SERVIDORES WEB HTTP, BASES DE DATOS**
Principales riesgos y prácticas de seguridad en aplicaciones web y sistemas de bases de datos.

Mensaje clave: la unidad construye una visión progresiva: conceptos base → vulnerabilidades → riesgo → propiedades de seguridad → seguridad web y de datos.

1.1.4 RESULTADO DE APRENDIZAJE DE LA UC 1

Resultado de aprendizaje

Comprender los fundamentos de la seguridad del software, identificando conceptos, vulnerabilidades, amenazas y riesgos que afectan al ciclo de vida del desarrollo, para analizar su impacto en la confiabilidad y resiliencia de los sistemas y aplicar directrices y estrategias de seguridad en hardware, código, diseño y arquitectura.



Fundamentos de seguridad

Comprender los conceptos esenciales de seguridad del software.

Vulnerabilidades

Identificar y analizar las vulnerabilidades en software, hardware, configuración y arquitectura.

Amenazas y riesgos

Reconocer las amenazas y evaluar su probabilidad e impacto en los sistemas.

Ciclo de vida

Considerar la seguridad en todas las fases del desarrollo, desde los requisitos hasta la operación.

Confiabilidad y resiliencia

Analizar cómo los riesgos afectan la confiabilidad y la capacidad de recuperación de los sistemas.

Estrategias de mitigación

Aplicar directrices y buenas prácticas de seguridad en hardware, código, diseño y arquitectura.

Competencias esperadas

Al finalizar la unidad, serás capaz de:

1. **Analizar** conceptos, amenazas, vulnerabilidades y riesgos en diferentes escenarios.
2. **Priorizar** los riesgos en función de su probabilidad e impacto.
3. **Proteger** aplicando controles y estrategias de seguridad adecuadas.
4. **Mejorar** la seguridad del software mediante decisiones informadas y prácticas continuas.

Mensaje clave: el objetivo de la unidad es pasar de la comprensión conceptual a la aplicación de controles de seguridad. (CISA, 2023; NIST, 2024)

La seguridad del software

es una disciplina que protege las aplicaciones a lo largo de todo su ciclo de vida para reducir vulnerabilidades y el riesgo operativo.



1. **Requisitos**
   Integrar requisitos de seguridad desde el inicio, considerando riesgos y cumplimiento.
2. **Diseño**
   Aplicar principios de seguridad y definir controles en la arquitectura y diseño.
3. **Implementación**
   Desarrollar código seguro, realizar revisiones y pruebas estáticas/dinámicas para detectar fallas.
4. **Despliegue**
   Configurar entornos de forma segura, gestionar dependencias y aplicar controles de liberación.
5. **Operación**
   Monitorear, detectar incidentes, gestionar vulnerabilidades y mantener mejoras continuas.

¿Por qué importa?

Reduce el riesgo de pérdidas económicas, interrupciones del servicio y daños a la confianza de los usuarios.

¿Qué se protege?

Activos del software: datos, servicios, procesos y reputación de la organización.

¿Frente a qué?

Amenazas como errores de desarrollo, ataques maliciosos, malas configuraciones y dependencias de terceros.

¿Cómo se logra?

Mediante controles de seguridad: prevención, detección, respuesta y recuperación.

Mensaje clave: La seguridad del software es transversal al ciclo de vida y no una actividad aislada.

(CISA, 2023; NIST, 2022)

Ciclo de vida del desarrollo de software (SDLC) con seguridad

La seguridad debe integrarse en todas las etapas del ciclo de vida para construir software más seguro y resiliente.



Beneficio de integrar seguridad temprano

Incorporar la seguridad desde las primeras etapas del SDLC proporciona importantes ventajas:

Menor coste
Corregir vulnerabilidades temprano es más económico.

Menos retrabajo
Se evitan cambios mayores en etapas avanzadas.

Menos exposición
Se reduce la probabilidad de incidentes de seguridad.

Mayor confiabilidad
Se construye software más seguro, estable y conidable.

Mensaje clave: integrar seguridad desde el inicio del SDLC reduce vulnerabilidades y costes de corrección. (NIST, 2022; CISA, 2023)

1.1 INTRODUCCIÓN — Amenazas, vulnerabilidades, riesgos y controles

1



Activo

Cualquier elemento de valor para la organización (datos, software, servicios, procesos o infraestructura).

2



Vulnerabilidad

Debilidad en el software, la configuración o el proceso que puede ser explotada por una amenaza.

3



Amenaza

Entidad, evento o condición que podría explotar una vulnerabilidad para causar un daño al activo.

4



Riesgo

Probabilidad de que ocurra una amenaza y el impacto que generaría sobre el activo si se explota la vulnerabilidad.

5



Control

Medida administrativa, técnica o física que reduce la probabilidad de que se explote la vulnerabilidad o el impacto del riesgo.

Ejemplo práctico: Formulario web público

Un formulario de contacto en un sitio web permite a los usuarios enviar mensajes.



Activo

Aplicación web y datos de los mensajes.



Vulnerabilidad

Validación de entrada débil (p. ej., campos sin sanitizar).



Amenaza

Atacante externo que envía entradas maliciosas (p. ej., inyección de código).



Riesgo

Acceso no autorizado o alteración de datos de la aplicación.



Control

Validación de entradas, WAF (Web Application Firewall) y consultas parametrizadas.

Relaciones clave



Exposición
Una vulnerabilidad expone al activo a una amenaza.



Impacto
El riesgo considera el posible daño sobre el activo.



Probabilidad
Depende de la existencia de la amenaza y de la facilidad de explotación de la vulnerabilidad.



Mitigación
Los controles reducen la probabilidad y/o el impacto del riesgo.



Mensaje clave: el riesgo aparece cuando una amenaza puede explotar una vulnerabilidad sobre un activo; los controles reducen probabilidad o impacto.

(NIST, 2024; OWASP, 2025)

1.1.1 QUÉ ES LA SEGURIDAD DEL SOFTWARE — Definición

Disciplina orientada a diseñar, desarrollar, desplegar y mantener software capaz de resistir ataques, errores y condiciones adversas, reduciendo vulnerabilidades y riesgo.



Seguridad del software

Propiedades y prácticas incorporadas al producto de software para prevenir, detectar y responder a amenazas.

Ciberseguridad

Protección del ecosistema digital frente a amenazas, ataques y riesgos que afecta a personas, organizaciones y sistemas.

Seguridad de aplicaciones

Protección de aplicaciones durante su ciclo de vida (desarrollo, pruebas, despliegue y ejecución) para evitar y mitigar vulnerabilidades.

Seguridad de la información

Protección de la confidencialidad, integridad y disponibilidad de la información, independientemente del medio o formato.

Idea clave: la seguridad debe diseñarse, no añadirse al final.

(Scarfone et al., 2022; CISA, 2023)

1.1.1 QUÉ ES LA SEGURIDAD DEL SOFTWARE — Relación con otras disciplinas



Seguridad del software

Se centra en el producto y su ciclo de vida.

Busca prevenir, identificar y mitigar vulnerabilidades a lo largo de todo el ciclo de vida del software (diseño, desarrollo, pruebas, despliegue y mantenimiento).



Ciberseguridad

Protege organizaciones, redes, servicios y usuarios.

Se enfoca en defender los sistemas de información, redes, infraestructura y usuarios frente a amenazas en el entorno digital, aplicando controles técnicos, administrativos y físicos.



Seguridad de aplicaciones (AppSec)

Prácticas y herramientas para proteger aplicaciones.

Conjunto de prácticas, estándares y herramientas orientadas a prevenir y detectar vulnerabilidades específicas en aplicaciones durante su desarrollo y operación.



Seguridad de la información

Preserva confidencialidad, integridad y disponibilidad de los datos.

Se enfoca en proteger la información de la organización, garantizando su confidencialidad, integridad y disponibilidad, independientemente del medio en el que se almacene o procese.

Puntos en común



Gestión de riesgos
Identificación, valoración y priorización.



Controles
Implementación de medidas técnicas, administrativas y físicas.



Monitoreo
Detección continua de amenazas y anomalías.



Respuesta
Acciones ante incidentas de seguridad.



Cumplimiento
Alineación con normas, estándares y regulaciones.

Ejemplo académico



Un sistema de matrículas requiere:

- **Código seguro** para evitar vulnerabilidades (seguridad del software).
- **Infraestructura protegida** para mantener la disponibilidad (ciberseguridad).
- **Pruebas de seguridad de aplicaciones (AppSec)** para detectar fallos.
- **Protección de los datos de estudiantes** (seguridad de la información).



Mensaje clave: la seguridad del software forma parte de un ecosistema más amplio, pero conserva un foco directo sobre el producto.

(NIST, 2024; OWASP, 2025)

1.1.1 QUÉ ES LA SEGURIDAD DEL SOFTWARE — Objetivos y beneficios

Objetivos principales

La seguridad del software busca gestionar los riesgos para:

1. **Prevenir**
   Evitar la introducción de vulnerabilidades y reducir la superficie de ataque.
2. **Detectar**
   Identificar oportunamente amenazas, vulnerabilidades y comportamientos anómalos.
3. **Responder**
   Contener y mitigar incidentes de seguridad de forma efectiva.
4. **Recuperarse**
   Restablecer el servicio y aprender de los incidentes para mejorar continuamente.

Reducir vulnerabilidades

Disminuir las debilidades en el software antes de su puesta en producción.

Proteger la información

Garantizar la confidencialidad, integridad y disponibilidad de los datos y transacciones.

Mantener la disponibilidad

Asegurar la continuidad del servicio y minimizar el tiempo de inactividad.

Aumentar la confianza

Generar confianza en usuarios, clientes y partes interesadas en el software y la organización.

Reducir costes

Disminuir la cantidad de incidentes, los gastos de recuperación y las posibles sanciones regulatorias.

Cumplir requisitos

Satisfacer normas, políticas, regulaciones y buenas prácticas de seguridad aplicables al software.

Beneficios para la organización

La seguridad del software aporta valor y fortalece el desempeño del negocio:

- **Mayor productidad**
  Menos interrupciones y mayor eficiencia en los procesos.
- **Mejor reputación**
  Mayor confianza de clientes, usuarios y socios estratégicos.
- **Mayor resiliencia**
  Capacidad de adaptarse y seguir operando ante amenazas y cambios del entorno.

Mensaje clave: Invertir en seguridad del software mejora confiabilidad, continuidad y valor para el negocio. (IBM, 2026; CISA, 2023)

1.1.1 QUÉ ES LA SEGURIDAD DEL SOFTWARE — Seguridad por diseño

Seguridad por diseño:

integrar requisitos, amenazas, controles y verificación desde las primeras fases del ciclo de vida.



Enfoque reactivo

La seguridad se aborda después de que ocurren los problemas.

Descubrir
Identificar vulnerabilidades después del despliegue.

Corregir
Aplicar parches y correcciones de emergencia.

Recuperar
Restaurar servicios y mitigar el impacto del incidente.

Mayor coste
Las correcciones suelen ser más caras.

Retrabajo
Se requiere rediseño y cambios en el código.

Presión de tiempo
Se trabaja bajo urgencia y con menos análisis.

Mayor exposición
Más tiempo expuesto a amenazas.

Enfoque preventivo

La seguridad se integra desde el inicio del ciclo de vida.

Diseñar
Integrar requisitos de seguridad y modelar amenazas desde el inicio.

Verificar
Comprobar controles y validar la seguridad durante el desarrollo y pruebas.

Monitorizar
Supervisar, detectar anomalías y mejorar de forma continua en producción.

Menor exposición
Se reducen las vulnerabilidades desde el inicio.

Controles tempranos
Se incorporan controles en todas las fases del ciclo de vida.

Automatización
Se integran pruebas y análisis automáticos de seguridad.

Mejora continua
Se aprende, se mide y se fortalecen los controles.

Prácticas clave:

1. **Modelado de amenazas**
   Identificar y analizar amenazas desde el diseño.
2. **Validación de entradas**
   Evitar datos no confiables y validar correctamente.
3. **Mínimo privilegio**
   Conceder solo los permisos necesarios.
4. **Gestión de dependencias**
   Controlar y actualizar librerías y componentes.
5. **Pruebas de seguridad**
   Aplicar análisis estático, dinámico y pruebas de penetración.
6. **Hardening**
   Configurar y endurecer el software, servidores y entornos.

Mensaje clave: diseñar seguridad desde el inicio reduce exposición, retrabajo y coste de corrección. (CISA, 2023; NIST, 2022)

1.1.2 LA IMPORTANCIA DE LA SEGURIDAD EN EL SOFTWARE — Panorama actual de amenazas



31%
de las brechas
explotación de vulnerabilidades como vector de acceso inicial.

↗ +55% interanual



48%
de las brechas
involucran a terceros / cadena de suministro.

↗ +60% interanual



40%
más tasa mediana de clics exitosos en phishing móvil frente al correo electrónico.



ENISA 2024
Amenazas principales: disponibilidad, ransomware y amenazas a los datos.



Explotación de vulnerabilidades
Aprovechamiento de fallas en software, configuraciones o componentes para obtener acceso no autorizado.



Ransomware
Cifrado de sistemas y datos para exigir un rescate, con alto impacto en la continuidad del negocio.



Credenciales / Phishing móvil
Robo de credenciales mediante ingeniería social, especialmente a través de dispositivos móviles.



Terceros / Cadena de suministro
Compromiso de proveedores y componentes de terceros que impacta en toda la cadena de valor.



Mensaje clave: El panorama actual combina explotación técnica, terceros y amenazas cada vez más automatizadas.

(Verizon, 2026; ENISA, 2024)

1.1.2 LA IMPORTANCIA DE LA SEGURIDAD EN EL SOFTWARE — Prevención vs. corrección



Seguridad reactiva

Se actúa **después** de que ocurre un incidente.

Detectar después → **Contener** → **Corregir** → **Recuperar**

Mayor exposición

Las vulnerabilidades permanecen activas por más tiempo.

Retrabajo

Se requieren esfuerzos adicionales para corregir.

Interrupción operativa

Puede afectar la disponibilidad del servicio.

Presión de tiempo

Las decisiones se toman bajo presión y con mayor riesgo de error.

VS



Seguridad preventiva

Se integran controles **desde el inicio** del ciclo de vida.

Diseñar → **Prevenir** → **Verificar** → **Monitorear**

Menor exposición

Se reducen las vulnerabilidades antes de que sean explotadas.

Controles tempranos

Se aplican buenas prácticas y controles desde el diseño.

Automatización

Se incorporan pruebas y análisis de seguridad automatizados.

Mejora continua

Se monitorea y evoluciona la seguridad durante todo el ciclo de vida.

Mensaje clave: Corregir vulnerabilidades antes de producción reduce exposición, retrabajo y riesgo. (Scarfone et al., 2022; CISA, 2023)

1.1.3 LAS VULNERABILIDADES Y SUS COSTES — Vulnerabilidad, amenaza, explotación e impacto

1



Activo

Aplicación web de e-commerce con catálogo de productos, usuarios y pagos.

2



Vulnerabilidad

La aplicación no valida correctamente las entradas del usuario en el formulario de búsqueda.

3



Amenaza

Un atacante externo con conocimientos técnicos busca encontrar y aprovechar esta debilidad.

4



Exploit

El atacante envía consultas SQL maliciosas a través del formulario de búsqueda para explotar la falla.

5



Incidente

El atacante logra ejecutar comandos en la base de datos y accede a información de clientes de la tienda.

6



Impacto

Se filtran datos de clientes (tarjetas y direcciones), la tienda pierde confianza, enfrenta sanciones y costes de recuperación.



Vulnerabilidades comunes

- Inyección SQL
- autenticación débil
- validación insuficiente
- configuraciones inseguras.



Motivaciones del atacante

- robo de datos
- fraude financiero
- extorsión
- daño reputacional.



Consecuencias

- pérdida de confidencialidad
- alteración de datos
- interrupción del servicio
- costes legales.



Prevención y mitigación

- buenas prácticas de desarrollo
- validación de entradas
- controles de acceso
- monitoreo.



Mensaje clave: Una vulnerabilidad no es el daño final; se convierte en riesgo cuando una amenaza puede explotarla.

(ENISA, 2024; Verizon, 2026)

1.1.3 LAS VULNERABILIDADES Y SUS COSTES — Origen de las vulnerabilidades

Requisitos	Diseño	Arquitectura	Código	Dependencias	Configuración	Infraestructura	Despliegue	Operación
Requisitos ambiguos: falta de claridad en los requisitos funcionales y de seguridad.	**Diseño sin threat modeling:** no se identifican amenazas ni se analizan riesgos.	**Arquitectura insegura:** patrones o decisiones que exponen superficies de ataque innecesarias.	**Código inseguro:** errores de programación (como inyección, desbordamientos o validación insuficiente).	**Librerías vulnerables:** uso de componentes de terceros con fallas conocidas.	**Mala configuración:** configuraciones por defecto o inadecuadas en aplicaciones, servicios y sistemas.	**Infraestructura expuesta:** servicios, puertos o recursos accesibles innecesariamente desde redes no confiables.	**Errores de despliegue:** versiones incorrectas, artefactos no verificados o exposición accidental.	**Fallas operativas:** monitoreo insuficiente, parcheo tardío, credenciales débiles y errores de administración.

Pueden aparecer en cualquier fase

Las vulnerabilidades pueden introducirse en todo el ciclo de vida.

Causas diversas

Provienen de errores humanos, decisiones técnicas, procesos y factores externos.

Se amplifican con el tiempo

Pueden incrementarse en impacto y explotarse si no se corrigen oportunamente.

Afectan a múltiples actores

Impactan a usuarios, organizaciones, socios y a la sociedad en general.

Requieren un enfoque preventivo

Deben incorporarse actividades de seguridad desde el inicio y de forma transversal.

Mensaje clave: Las vulnerabilidades pueden introducirse en múltiples fases del ciclo de vida; por eso la seguridad debe ser transversal.

(Scarfone et al., 2022; CISA, 2023)

1.1.3 LAS VULNERABILIDADES Y SUS COSTES — Vulnerabilidades frecuentes



Control de acceso deficiente
acceso no autorizado a funcionalidades o datos.



Inyección
ejecución de comandos o consultas no autorizadas por entradas no validadas.



Autenticación insegura
suplantación de identidad y acceso no autorizado.



Fallos criptográficos
uso incorrecto de cifrado, algoritmos o claves.



Configuración insegura
ajustes por defecto o mal configurados.



Componentes vulnerables
librerías o dependencias sin parches.



SSRF
solicitudes no autorizadas desde el servidor.



Exposición de secretos
claves, tokens o credenciales expuestos.



Vulnerabilidades de API
falta de validación, control de acceso o límites.



Mensaje clave: Los errores más comunes combinan fallos de validación, acceso, configuración y gestión de componentes.



Estas categorías se alinean con prácticas y controles de verificación de OWASP ASVS 5.0.

(OWASP, 2025)

1.1.3 LAS VULNERABILIDADES Y SUS COSTES — Del CVE al ataque



Conceptos clave

- **CVE:** identificador único para una vulnerabilidad conocida.
- **CWE:** clasificación de tipos de debilidades de software.
- **CVSS:** sistema para asignar severidad a una vulnerabilidad.
- **Zero-day:** vulnerabilidad desconocida para el fabricante, sin parche disponible.
- **Vulnerabilidad explotada conocida:** vulnerabilidad pública con exploit disponible.

Principales costes de un incidente

- **Costes directos:** gastos de respuesta, recuperación, parches, consultoría y posibles multas.
- **Pérdida de productividad:** interrupción de servicios y tiempo de inactividad.
- **Daño reputacional:** pérdida de confianza de clientes, socios y usuarios.
- **Consecuencias legales:** sanciones regulatorias, demandas y responsabilidades por incumplimiento.

Buenas prácticas para reducir el riesgo

- **Inventario:** mantener un inventario actualizado de activos y componentes de software.
- **Priorización:** evaluar la severidad (CVSS), la exposición y el contexto del negocio para priorizar la remediación.
- **Parcheo y mitigación:** aplicar parches de forma oportuna y usar mitigaciones (configuración, detección, segmentación, etc.).

Idea clave: Cuanto menor sea el tiempo entre la divulgación de una vulnerabilidad y su explotación, mayor es el riesgo y el coste potencial.

(MITRE, 2026; NIST NVD, 2026; CISA, 2023)

1.1.3 LAS VULNERABILIDADES Y SUS COSTES — Coste económico y organizacional

USD 4.99M
coste promedio global de una brecha.
2024 data reported in 2026 | IBM

+12%
incremento frente al año anterior.
2026 | IBM

USD 1.93M
ahorro por uso extensivo de IA y automatización en seguridad.
2026 | IBM

USD 6M
coste promedio de un ataque de model inversion en IA.
2026 | IBM

Detección y escalamiento
Costes de monitorización, investigación forense y respuesta a incidentes.

Pérdida operativa
Interrupción del negocio, pérdida de productividad y reducción de ingresos.

Afectación reputacional
Pérdida de confianza, daño a la marca y disminución de oportunidades de negocio.



Costes regulatorios
Multas, sanciones y mayores requisitos de cumplimiento normativo.

Recuperación
Restauración de sistemas, mejora de controles y fortalecimiento de la seguridad.

Impacto sobre clientes
Pérdida de datos, interrupción de servicios y disminución de la satisfacción.

Mensaje clave: El impacto de una brecha combina costes directos, interrupción del negocio y pérdida de confianza. (IBM, 2026)

1.1.4 GESTÍÓN DE RIESGOS DE SEGURIDAD DEL SOFTWARE — Concepto de riesgo

Riesgo = probabilidad × impacto

Activo
Recurso valioso que debe protegerse (ej. datos, sistemas, servicios).

+

Amenaza
Fuente potencial de daño (ej. atacante, malware, desastre natural).

+

Vulnerabilidad
Debilidad que puede ser explotada (ej. fallo de configuración, código inseguro).

+

Exposición
Situación que permite que la amenaza explote la vulnerabilidad (ej. servicio expuesto a Internet).

→

Riesgo
Posibilidad de que ocurra un incidente y su consecuencia (probabilidad × impacto).

¿Qué es el riesgo?

Posibilidad de que una amenaza explote una vulnerabilidad de un activo y cause un impacto en los objetivos de la organización.

Riesgo inherente

- Riesgo antes de aplicar controles.
- Refleja la exposición natural del activo.
- Suele ser mayor.

Riesgo residual

- Riesgo después de aplicar controles.
- Refleja el riesgo que aún permanece.
- Puede aceptarse, monitorearse o tratarse.

Ejemplo: API crítica con autenticación débil

- **Activo:** API de pagos.
- **Amenaza:** atacante externo.
- **Vulnerabilidad:** autenticación débil.
- **Exposición:** API expuesta a Internet.
- **Riesgo:** acceso no autorizado y fraude en transacciones.

Los controles de seguridad **reducen la probabilidad** de explotación, **el impacto o ambos.**
El riesgo residual debe **aceptarse, monitorearse o tratarse,** según el apetito de riesgo de la organización.

Mensaje clave: El riesgo expresa la posibilidad de que una amenaza explote una vulnerabilidad y genere impacto. (NIST, 2024)

1.1.4 GESTIÓN DE RIESGOS DE SEGURIDAD DEL SOFTWARE — Proceso de gestión

Enfoque sistemático
Proporciona un proceso estructurado para gestionar los riesgos de ciberseguridad.

Mejora la toma de decisiones
Facilita decisiones basadas en el contexto, los riesgos y los objetivos del negocio.

Aplicable a cualquier organización
Flexible y escalable para diferentes sectores, tamaños y niveles de madurez.



Fortalece la resiliencia
Ayuda a anticipar, absorber, responder y recuperarse de eventos de ciberseguridad.

Enfoque basado en riesgos
Prioriza esfuerzos según el análisis y la evaluación de riesgos.

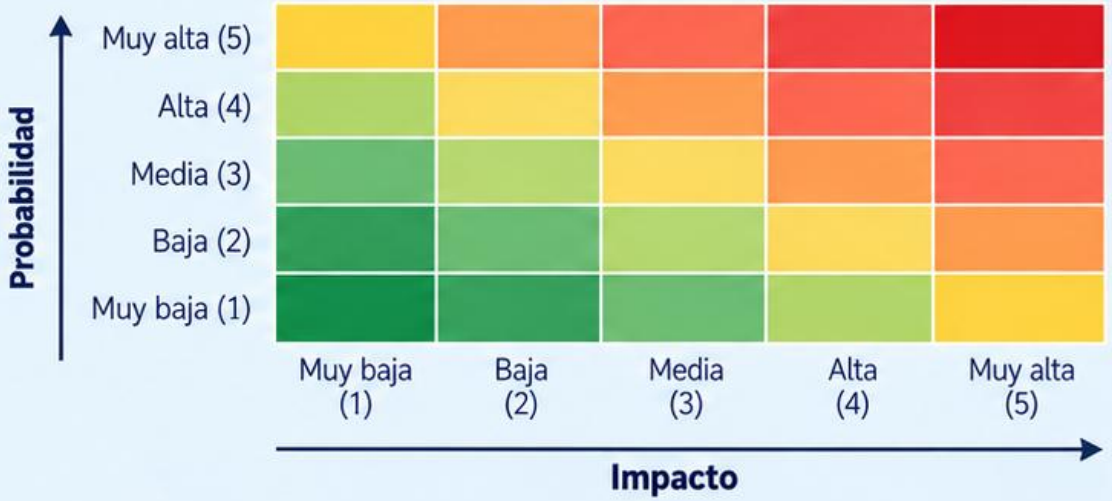
Marco reconocido
Desarrollado por el NIST y adoptado globalmente como buena práctica.

Mensaje clave: El NIST CSF 2.0 proporciona un marco para que las organizaciones mejoren la gestión de riesgos de ciberseguridad de forma continua, adaptativa y alineada con sus objetivos. (NIST, 2024)

1.1.4 GESTIÓN DE RIESGOS DE SEGURIDAD DEL SOFTWARE — Identificación y análisis

1. **Identificar activos**
   Determinar los activos de software, datos, servicios y procesos críticos.
2. **Identificar amenazas**
   Reconocer las fuentes de amenaza que podrían explotar los activos.
3. **Identificar vulnerabilidades**
   Encontrar debilidades en el software, su configuración o entorno.
4. **Estimar probabilidad**
   Evaluar la posibilidad de que la amenaza explote la vulnerabilidad.
5. **Estimar impacto**
   Analizar las consecuencias en caso de que ocurra el evento de seguridad.
6. **Determinar nivel de riesgo**
   Combinar probabilidad e impacto para asignar el nivel de riesgo.

Matriz de riesgo: Probabilidad × Impacto (5 × 5)



Visión integral del riesgo
Permite comprender el panorama completo de amenazas, vulnerabilidades y su posible impacto en los objetivos del sistema.

Prioriza esfuerzos
Facilita enfocar los recursos en los riesgos más relevantes y tratar primero los escenarios de mayor probabilidad e impacto.

Soporta la toma de decisiones
Proporciona información objetiva para seleccionar y justificar controles de seguridad adecuados.

Proceso iterativo
Debe repetirse periódicamente o ante cambios en el contexto, para mantener el análisis de riesgo actualizado y relevante.

Mensaje clave: La priorización del riesgo permite asignar controles y recursos donde más se necesitan. (NIST, 2024)

1.1.4 GESTIÓN DE RIESGOS DE SEGURIDAD DEL SOFTWARE — Tratamiento del riesgo

Tratar el riesgo = decidir qué hacer con el riesgo identificado

Una vez evaluado el nivel de riesgo, se selecciona la respuesta más adecuada de acuerdo con los objetivos del negocio, los recursos disponibles y el apetito de riesgo de la organización.

1. **Mitigar**
   Reducir la probabilidad o el impacto del riesgo mediante la implementación de controles de seguridad.
2. **Transferir**
   Compartir el riesgo con terceros, por ejemplo, mediante seguros, contratos o outsourcing.
3. **Aceptar**
   Asumir el riesgo cuando el riesgo residual es tolerable y está dentro del apetito de riesgo.
4. **Evitar**
   Eliminar la actividad que genera el riesgo o cambiar el diseño para eliminar o reducir la amenaza.

Criterios para decidir

La selección de la respuesta debe considerar:

- Criticidad del activo.
- Apetito de riesgo de la organización.
- Coste-beneficio de las opciones.
- Requisitos de cumplimiento.

Ejemplo

Riesgo identificado: Autenticación débil en una API expuesta a Internet.

- **Mitigar:** Implementar MFA y limitación de tasa (rate limiting).
- **Transferir:** Incluir cláusulas de seguridad con el proveedor del servicio.
- **Aceptar:** Si el riesgo residual es bajo y tolerable.
- **Evitar:** No exponer la función o rediseñar la API.

Resultado esperado

Un tratamiento adecuado del riesgo permite:

- Menor exposición a amenazas.
- Controles priorizados y más eficientes.
- Responsabilidad clara sobre la gestión del riesgo.

Mensaje clave: Tratar el riesgo implica seleccionar la respuesta más adecuada según el contexto del negocio y el nivel de riesgo.

(NIST, 2024; CISA, 2023)

1.1.4 GESTIÓN DE RIESGOS DE SEGURIDAD DEL SOFTWARE — Monitoreo y mejora continua

1. **Monitorear controles**
   Supervisar el funcionamiento de los controles de seguridad implementados.
2. **Vigilar vulnerabilidades**
   Mantener un seguimiento continuo de nuevas vulnerabilidades en software, infraestructura y terceros.
3. **Detectar eventos**
   Identificar y analizar eventos e incidentes de seguridad relevantes en tiempo oportuno.
4. **Medir resultados**
   Evaluar la efectividad de los controles mediante métricas y comparación con los objetivos.
5. **Ajustar controles**
   Realizar cambios en la configuración, procesos o controles según los resultados obtenidos.
6. **Aprender y mejorar**
   Incorporar lecciones aprendidas y buenas prácticas para fortalecer continuamente la gestión de riesgos.

Fuentes de información

Recopilar y consolidar información relevante de múltiples fuentes:

- Logs de sistemas y aplicaciones
- Plataformas SIEM
- Escáneres de vulnerabilidades
- Sistema de tickets y gestión de incidentes
- Auditorías y revisiones de cumplimiento

Indicadores

Utilizar métricas para evaluar el estado del riesgo y la efectividad de los controles:

- MTTD (tiempo medio de detección)
- MTTR (tiempo medio de respuesta/ recuperación)
- Vulnerabilidades abiertas y su tendencia
- Cobertura de parches
- Cumplimiento de controles y políticas

Retroalimentación

Incorporar información del entorno operativo para mejorar la gestión de riesgos:

- Incidentes reales y casi incidentes
- Resultados de pruebas de seguridad
- Ejercicios de pentesting y red teaming
- Lecciones aprendidas
- Opiniones de equipos operativos y usuarios

Beneficios

El monitoreo y la mejora continua permiten obtener:

- Reducción del riesgo residual
- Detección y respuesta más rápida
- Mayor resiliencia del sistema
- Cumplimiento continuo de requisitos normativos y de negocio
- Cultura de seguridad basada en datos

Mensaje clave: La gestión de riesgos no termina con la decisión inicial; requiere seguimiento continuo, métricas y mejora iterativa. (NIST, 2024; CISA, 2023)

1.1.5 PROPIEDADES DEL SOFTWARE SEGURO Y RESILIENTE — Propiedades esenciales

Un software seguro y resiliente
protege activos, mantiene funciones críticas y se recupera ante fallos o ataques.

Confidencialidad

Solo las personas, servicios o procesos autorizados pueden acceder a la información y a los recursos.

Integridad

Los datos, configuraciones y funciones no se alteran de forma indebida o no autorizada.

Disponibilidad

Los servicios y funciones están accesibles y operativos cuando se necesitan.

Autenticidad

Las identidades, orígenes de la información y componentes son verificables y confiables.

Trazabilidad

Las acciones, eventos y cambios se registran de forma completa, con suficiente contexto para auditoría y análisis.

Resiliencia

Capacidad para resistir, adaptarse y recuperarse ante fallos, ataques o eventos inesperados, manteniendo las funciones críticas.

Diseño seguro

- Aplicar principios de seguridad desde el inicio.
- Minimizar superficies de ataque.
- Usar componentes confiables y actualizados.

Controles en operación

- Implementar controles técnicos y organizativos.
- Monitorear y detectar incidentes.
- Gestionar vulnerabilidades y cambios.

Recuperación

- Contar con planes de continuidad.
- Realizar copias de seguridad.
- Probar la recuperación regularmente.

Mensaje clave: La seguridad no es una sola característica; es el conjunto de propiedades que permite confiar en el sistema.

(CISA, 2023; OWASP, 2025)

1.1.5 PROPIEDADES DEL SOFTWARE SEGURO Y RESILIENTE — Confidencialidad, Integridad y Disponibilidad

Confidencialidad

Garantiza que la información solo sea accesible por personas, sistemas o procesos autorizados.

Ejemplos:
cifrado, control de acceso, gestión de secretos.

Integridad

Asegura que la información se mantenga completa, correcta y no sea modificada de manera no autorizada.

Ejemplos:
hashes, validación, control de cambios.

Disponibilidad

Garantiza que la información y los servicios estén disponibles y accesibles cuando se necesiten.

Ejemplos:
redundancia, copias de seguridad, tolerancia a fallos.



Las tres propiedades se complementan

Confidencialidad, integridad y disponibilidad están interrelacionadas. Mejorar una puede afectar a las otras, por lo que es importante equilibrarlas según el contexto y los requisitos del sistema.

Sistema académico

Un sistema académico debe mantener confidenciales las calificaciones y datos de los estudiantes, garantizar su integridad (sin modificaciones no autorizadas) y estar disponible para docentes y estudiantes cuando se necesite.

Amenazas que afectan la CIA

Filtración de datos
Acceso no autorizado a información sensible (confidencialidad).

Manipulación de datos
Modificación no autorizada de información (integridad).

Ransomware
Cifra la información y bloquea el acceso a los servicios (disponibilidad).

Ataques DDoS
Saturan los servicios e impiden su funcionamiento (disponibilidad).

Eliminación accidental
Pérdida de información por errores humanos o fallos operativos (integridad y disponibilidad).

Mensaje clave: La tríada CIA sigue siendo la base para evaluar y proteger la información y los servicios del software.

(NIST, 2024; CISA, 2023)

Resiliencia = capacidad de anticipar, resistir, continuar operando y recuperarse

Habilidad del sistema para mantener o restablecer sus funciones ante incidentes, perturbaciones o cambios, minimizando el impacto en los usuarios, los datos y los objetivos del negocio.

1. **Anticipar**
   Identificar posibles amenazas, dependencias y escenarios de falla.
2. **Resistir**
   Prevenir o limitar efectivamente que las perturbaciones afecten el sistema.
3. **Absorber**
   Mantener la operación ante la degradación, conteniendo el impacto y aislando fallas.
4. **Recuperar**
   Restablecer los servicios a niveles normales en el menor tiempo posible y de forma controlada.
5. **Adaptar**
   Aprender de la experiencia y mejorar el sistema, fortaleciendo su capacidad de respuesta futura.

Mecanismos

Técnicas que aumentan la resiliencia del sistema:

- Redundancia
- Backups
- Failover (conmutación)
- Segmentación

Buenas prácticas

Actividades que fortalecen la resiliencia:

- Pruebas de recuperación
- Planes de continuidad
- Observabilidad y monitoreo
- Gestión de dependencias

Indicadores

Métricas para evaluar la resiliencia:

- RTO (Recovery Time Objective)
- RPO (Recovery Point Objective)
- Disponibilidad del servicio
- Tiempo de recuperación (TTR)

Ejemplo

Servicio web que se degrada gradualmente mientras restaura la base de datos.

Ante una falla en la base de datos, el servicio continúa operando en modo solo lectura y muestra información en caché, mientras se ejecuta la restauración, volviendo al servicio completo una vez recuperada la base de datos.

Mensaje clave: Un sistema resiliente no evita todos los incidentes, pero reduce el impacto y acelera la recuperación. (NIST, 2024; CISA, 2023)

1.1.6 CONCEPTOS DE SEGURIDAD — Amenaza, vulnerabilidad, riesgo y control



Activo

Recurso de valor para la organización (datos, software, servicios, equipos o procesos).



Vulnerabilidad

Debilidad en el software, la configuración o los procesos que puede ser explotada por una amenaza.



Amenaza

Fuente de peligro que puede explotar una vulnerabilidad y causar daño al activo.



Riesgo

Combinación de la probabilidad de que ocurra una amenaza y el impacto que generaría sobre el activo.

Reduce



Control

Medida administrativa, técnica o física que reduce la probabilidad de ocurrencia de una amenaza o su impacto.



Exposición

Situación en la que un activo queda accesible a una amenaza, generalmente debido a su ubicación, configuración o uso.



Incidente

Evento que indica que una amenaza se ha materializado y ha afectado uno o más activos.



Impacto

Consecuencia negativa que resulta de un incidente sobre los activos (pérdida de confidencialidad, integridad, disponibilidad, costos, etc.).

Ejemplo práctico: Formulario web público con validación débil



Activo

Aplicación web y base de datos de clientes.



Vulnerabilidad

Validación de entrada deficiente en el formulario de contacto.



Amenaza

Atacante que envía consultas SQL maliciosas (SQL Injection).



Riesgo

Acceso no autorizada, exfiltración o modificación de datos de clientes.



Control

Uso de consultas parametrizadas y un WAF (Web Application Firewall) para bloquear intentos de inyección y reducir la probabilidad o el impacto del ataque.



Mensaje clave: El riesgo surge cuando una amenaza puede explotar una vulnerabilidad sobre un activo; los controles se aplican para reducir esa posibilidad o su impacto.

(NIST, 2024; OWASP, 2025)

1.1.6 CONCEPTOS DE SEGURIDAD — Principios de seguridad

Los principios orientan decisiones de arquitectura, desarrollo, despliegue y operación.

1

Mínimo privilegio

Conceder solo los permisos necesarios para realizar una tarea y durante el tiempo requerido.

2

Defensa en profundidad

Aplicar múltiples capas de controles para reducir la probabilidad de éxito de un ataque.

3

Segregación de funciones

Dividir responsabilidades para evitar que una sola persona tenga control completo sobre procesos críticos.

4

Autenticación multifactor

Requerir dos o más factores de autenticación para verificar la identidad del usuario.

5

Zero Trust

No confiar por defecto en ningún usuario, dispositivo o componente, verificando y autorizando cada solicitud.

6

Hardening

Configurar y reducir la superficie de ataque eliminando servicios innecesarios, aplicando configuraciones seguras y parches actualizados.

7

Fail secure

Ante un fallo o condición anómala, el sistema debe pasar a un estado seguro por defecto (denegar el acceso) en lugar de uno abierto.

8

Seguridad por diseño

Integrar la seguridad desde las fases iniciales del ciclo de vida, teniendo en cuenta amenazas, requisitos y controles en el diseño.

Aplicación práctica

En APIs y microservicios: aplicar mínimo privilegio en cuentas de servicio, validar identidad en cada servicio (Zero Trust), usar mTLS o tokens con MFA, segmentar redes, hardening de contenedores y diseñar con seguridad desde el inicio.

Beneficio

- Reducción de la superficie de ataque.
- Contención de incidentes.
- Mejor trazabilidad y auditoría.

Mensaje clave: Los principios de seguridad proporcionan reglas de diseño para construir software más robusto y menos expuesto. (CISA, 2023; OWASP, 2025)

1.1.7 SERVIDORES WEB HTTP, BASES DE DATOS — Servidor web y protocolo HTTP



Cliente
Navegador u otra aplicación que envía una solicitud HTTP.

Solicitud HTTP

```
GET /pagina HTTP/1.1
Host: www.ejemplo.com
User-Agent: ...
...
```



Servidor web
Recibe la solicitud, la procesa y genera una respuesta.

Respuesta HTTP

```
HTTP/1.1 200 OK
Content-Type: text/html
Content-Length: ...
...
```



Respuesta HTTP
El servidor envía contenido y cabeceras en la respuesta.



Servidor web

Recibe solicitudes HTTP, sirve contenido (páginas web, archivos, API) y enruta el tráfico hacia los recursos adecuados.



HTTP

Protocolo de aplicación basado en solicitudes y respuestas entre cliente y servidor.



Métodos comunes

GET: obtener un recurso
POST: enviar datos
PUT: actualizar un recurso
DELETE: eliminar un recurso



Códigos de estado

200	**OK**: solicitud exitosa.
301	Movido permanentemente.
404	No encontrado.
500	Error interno del servidor.



HTTPS

HTTP sobre TLS (Transport Layer Security). Proporciona confidencialidad e integridad de los datos entre el cliente y el servidor.



Riesgos frecuentes

- Cabeceras inseguras.
- Exposición de información sensible.
- Configuraciones débiles del servidor.
- Falta de uso de TLS (HTTP en claro).



Mensaje clave: Comprender el flujo HTTP y la función del servidor web es esencial para identificar riesgos y aplicar configuraciones seguras.

(RFC 9110, 2022; OWASP, 2025)

1.1.7 SERVIDORES WEB HTTP, BASES DE DATOS — Seguridad en bases de datos



1 Control de acceso

Gestionar usuarios y roles, aplicar el principio de mínimo privilegio y separar funciones administrativas.

- Usuarios y roles
- Mínimo privilegio
- Segregación de funciones



2 Protección de datos

Cifrar los datos en tránsito y en reposo, y aplicar mascarado o tokenización para datos sensibles.

- Cifrado en tránsito (TLS)
- Cifrado en reposo
- Mascarado de datos sensibles



3 Consultas seguras

Usar consultas parametrizadas y procedimientos almacenados para prevenir la inyección SQL y validar las entradas.

- Consultas parametrizadas
- Procedimientos almacenados
- Validación de entradas
- Evitar inyección SQL





4 Disponibilidad

Implementar copias de seguridad (backups), replicación y planes de recuperación ante desastres.

- Copias de seguridad periódicas
- Replicación (primaria-secundaria)
- Pruebas de restauración
- Plan de recuperación ante desastres



5 Monitoreo y auditoría

Registrar y analizar eventos relevantes, configurar alertas de actividad sospechosa y garantizar la trazabilidad.

- Logs de acceso y cambios
- Alertas de actividad inusual
- Auditoría de acciones críticas
- Trazabilidad y retención de registros



6 Hardening

Mantener el sistema actualizado, aplicar configuración segura y desactivar servicios y funcionalidades innecesarias.

- Parches y actualizaciones
- Configuración segura
- Desactivar servicios innecesarios
- Eliminar cuentas por defecto
- Revisar permisos y accesos



Ejemplo: Sistema académico

La base de datos almacena información de estudiantes, calificaciones y pagos, por lo que requiere controles estrictos de seguridad para proteger datos personales y financieros.



Datos de estudiantes
- Información personal
- Datos de contacto
- Historial académico



Calificaciones
- Cursos y notas
- Estado académico
- Registro histórico



Pagos
- Matrículas y pensiones
- Transacciones
- Comprobantes de pago



Mensaje clave: La seguridad de la base de datos combina control de acceso, protección de datos, consultas seguras y capacidad de recuperación.

(NIST, 2024; OWASP, 2025)

1.1.7 SERVIDORES WEB HTTP, BASES DE DATOS — Arquitectura web segura

Arquitectura web segura y controles por capa



Principios transversales

Defensa en profundidad
Múltiples capas de control para reducir el riesgo.

Segregación
Separar funciones, entornos y responsabilidades.

Observabilidad
Monitoreo, registros y alertas para detectar y responder.

Flujo seguro

1. **Solicitud** — El cliente envía la petición.
2. **Inspección** — El WAF aplica políticas de seguridad.
3. **Procesamiento** — El servidor web y la aplicación procesan la solicitud.
4. **Persistencia** — Se accede a la base de datos de forma segura.
5. **Auditoría** — Se registran eventos para trazabilidad y respuesta.

Mensaje clave: La seguridad web efectiva se logra aplicando controles coordinados en todas las capas de la arquitectura. (CISA, 2023; OWASP, 2025)

1.1.7 UNIDAD 1 — Síntesis integradora y conclusiones

De los fundamentos a la protección integral del software

La unidad articula conceptos, vulnerabilidades, riesgos, propiedades de seguridad y controles en aplicaciones web y bases de datos.

1 Fundamentos



Activos, amenazas, vulnerabilidades y controles.

2 Identificar



Reconocer debilidades en requisitos, diseño, código, despliegue y operación.

3 Evaluar riesgos



Analizar probabilidad e impacto para priorizar acciones.

4 Proteger



Aplicar seguridad por diseño, mínimo privilegio, hardening y validación.

5 Asegurar propiedades



Confidencialidad, integridad, disponibilidad y resiliencia.

6 Operar y mejorar



Monitoreo, respuesta, recuperación y mejora continua.

Competencias logradas

- **Comprender** conceptos clave de seguridad del software.
- **Distinguir** vulnerabilidades, amenazas y riesgos.
- **Priorizar** controles según el nivel de riesgo.
- **Relacionar** principios de seguridad con la arquitectura web y de datos.
- **Proponer** medidas preventivas y correctivas en el SDLC.

Ideas fuerza

- La seguridad debe integrarse desde el inicio.
- Toda vulnerabilidad puede convertirse en riesgo si existe una amenaza y exposición.
- Los controles reducen probabilidad, impacto o ambos.
- La resiliencia permite resistir y recuperarse.

Aplicación práctica



En una aplicación web segura se combinan: validación de entradas, autenticación robusta, WAF o reverse proxy, TLS, consultas parametrizadas, cifrado, backups y monitoreo.

Proyección



Esta base conceptual prepara el estudio posterior de prácticas, herramientas y evaluaciones de seguridad en escenarios reales de desarrollo.

Mensaje clave: La seguridad del software es un proceso continuo que conecta diseño, prevención, operación y mejora para proteger sistemas, datos y servicios.

(NIST, 2024; CISA, 2023; OWASP, 2025)

Fuentes clave utilizadas en la unidad

para conceptos, riesgos, vulnerabilidades, resiliencia, seguridad web y bases de datos.



Marcos y estándares

1. **National Institute of Standards and Technology.** (2022). *Secure software development framework (SSDF) version 1.1: Recommendations for mitigating the risk of software vulnerabilities (SP 800-218).* NIST. https://csrc.nist.gov/
2. **National Institute of Standards and Technology.** (2024). Cybersecurity framework (CSF) 2.0. NIST. https://www.nist.gov/cyberframework
3. **RFC Editor.** (2022). *RFC 9110: HTTP semantics.* https://www.rfc-editor.org/

Buenas prácticas y guías

1. **Cybersecurity and Infrastructure Security Agency.** (2023). Shifting the balance of cybersecurity risk: Principles and approaches for secure by design software. CISA. https://www.cisa.gov/
2. **OWASP Foundation.** (2025). Application security verification standard (ASVS) 5.0. OWASP. https://owasp.org/
3. **OWASP Foundation.** (2025). OWASP Top 10. OWASP. https://owasp.org/

Reportes y panoramas

1. **European Union Agency for Cybersecurity.** (2024). *ENISA threat landscape 2024.* ENISA. https://www.enisa.europa.eu/
2. **IBM Security.** (2026). Cost of a data breach report 2026. IBM. https://www.ibm.com/
3. **Verizon.** (2026). Data breach investigations report 2026. Verizon. https://www.verizon.com/business/resources/reports/dbir/

Recursos de vulnerabilidades

1. **National Institute of Standards and Technology.** (2024). National Vulnerability Database (NVD). NIST. https://nvd.nist.gov/
2. **MITRE Corporation.** (2026). Common vulnerabilities and exposures (CVE). https://www.cve.org/

Uso en la unidad

- **Definiciones y principios** de seguridad.
- **Gestión de riesgos** y resiliencia.
- **Vulnerabilidades, controles** y AppSec.
- **Seguridad en HTTP,** aplicaciones web y bases de datos.

Mensaje clave: La formación en seguridad del software debe sustentarse en estándares, guías y reportes actualizados y verificables.